

CS 816 Mini Project Report

Scientific Calculator with DevOps Pipeline

Author : Swayam Palrecha

Roll No. : MT2025126

1 . Introduction

1.1 Scientific Calculator Overview

This project presents a menu-driven Command Line Interface (CLI) based scientific calculator designed to perform essential mathematical operations such as square root, factorial, natural logarithm, and power functions. The application is lightweight, efficient, and easy to execute using a terminal interface.

1.2 DevOps Overview

DevOps is a modern software engineering approach that integrates software development and IT operations to automate and streamline the software lifecycle. It focuses on continuous integration, automated testing, reliable deployment, and real-time monitoring to ensure faster and more stable software delivery.

1.3 Why DevOps?

- **Speed:** Enables faster software development and deployment so teams can innovate quickly and respond to changing requirements.
- **Rapid Delivery:** Increases the frequency of releases, allowing continuous improvement of the application.
- **Reliability:** Ensures stable and high-quality software through automated testing and consistent deployment processes.
- **Automation:** Reduces manual effort by automating build, test, integration, and deployment workflows.

- Scalability: Helps manage infrastructure and applications efficiently across different environments.
- Continuous Feedback: Provides real-time monitoring and logging to quickly detect and fix issues.

2. Project Objectives

- To develop a menu-driven Command Line Interface (CLI) based scientific calculator.
- To implement the complete DevOps lifecycle using modern automation tools and practices.
- To automate the processes of building, testing, integration, deployment, and monitoring.
- To containerize the application using Docker for consistent and portable execution across environments.

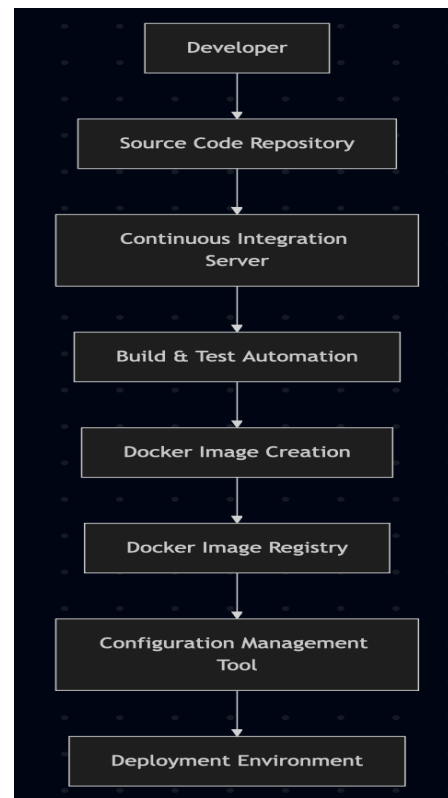
3. Technology Stack (DevOps Toolchain)

Stage	Tool Used	Description
Source Control	Github	Used to store source code, manage version history, and enable collaboration.
Testing	JUnit	Used to write and execute automated unit tests for validating calculator functions.
Build	Maven	Used to compile source code, manage dependencies, and package the application.
Ci/CD	Jenkins	Automates build and testing whenever code changes are pushed to

		the repository.
Containerization	Docker	Packages the application and its dependencies into portable containers.
Image Registry	Docker Hub	Cloud repository used to store and distribute Docker images.
Configuration Management	Anisble	Automates deployment and manages infrastructure configuration.

4. System Architecture

- The developer writes and updates the scientific calculator source code.
- The code is stored in a source repository, which containerized CI server on every update.
- The CI server automates build and testing to verify code quality and functionality.
- A Docker image is created and stored in a registry for consistent and portable deployment.
- A configuration management tool deploys the containerized application to the target environment.



5. Pipeline Stages Explanation

The CI/CD pipeline automates the complete build, test, containerization, and deployment lifecycle of the Scientific Calculator application. Each stage performs a specific task and ensures quality before moving to the next step.

5.1 Checkout SCM

- Jenkins connects to the source code repository.
- The latest version of the project is downloaded to the Jenkins workspace.
- Ensures the pipeline runs on the most recent committed code.

5.2 Prepare Docker Config

- Required Docker configuration files are prepared.
- The Dockerfile and environment settings are validated.
- Ensures the container build process is correctly configured.

5.3 Build and Test with Maven

- Source code is compiled into executable bytecode.
- Dependencies defined in pom.xml are automatically installed.
- Unit tests are executed to verify calculator functionality.
- The pipeline stops if compilation or tests fail.

5.4 Build Docker Image

- Jenkins builds a Docker image using the Dockerfile.
- The application and runtime environment are packaged together.
- Ensures consistent execution across all systems.

5.5 Login to DockerHub

- Jenkins securely authenticates with Docker Hub.
- Credentials are used to enable image upload.

5.6 Push Docker Image

- The Docker image is uploaded to Docker Hub.

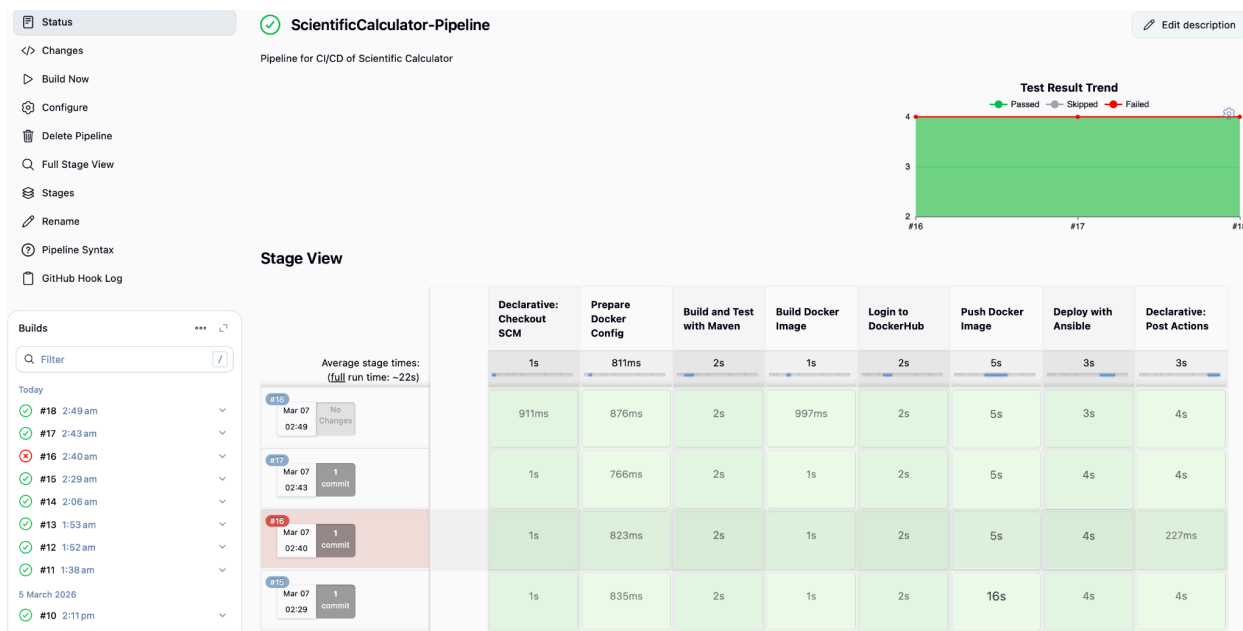
- The image becomes globally accessible for deployment.
- Maintains version history for tracking releases.

5.7 Deploy with Ansible

- Ansible connects to the target machine.
- The latest Docker image is pulled from Docker Hub.
- A new container is started to run the application.
- Ensures automated and consistent deployment.

5.8 Post Actions

- Cleanup tasks are performed after pipeline execution.
- Logs and results are stored for monitoring and auditing.
- Notifications will be sent for success or failure.



6. Final Result and Operation

The Scientific Calculator application was successfully built, tested, containerized, and deployed using the automated DevOps pipeline. Since the application is CLI-based, it runs directly in the terminal instead of a web browser.

Users can execute the calculator using Docker containers or by running the application locally from the source code.

Option A: Run Using Docker (Recommended)

Pull Docker Image

```
docker pull swayam614/scientific-calculator:latest
```

Run Container

```
docker run -it swayam614/scientific-calculator:latest
```

```
> docker pull swayam614/scientific-calculator:latest
latest: Pulling from swayam614/scientific-calculator
Digest: sha256:a017712be11a4c3d507f22cbe59a13e1638c8e2fd511d11525f571cff3827ec8
Status: Image is up to date for swayam614/scientific-calculator:latest
docker.io/swayam614/scientific-calculator:latest
> docker run -it swayam614/scientific-calculator:latest

Scientific Calculator
1. Square Root ( $\sqrt{x}$ )
2. Factorial ( $x!$ )
3. Natural Log ( $\ln x$ )
4. Power ( $x^b$ )
5. Exit
Choose an option: 5
Exiting Calculator...
```

The calculator menu appears in the terminal, allowing users to perform mathematical operations interactively.

Option B: Run Locally Using Source Code

Clone Repository

```
git clone
```

```
https://github.com/swayam614/ScientificCalculator-Devops.git
```

```
cd ScientificCalculator-Devops
```

Build Application

```
mvn clean package
```

Run Application

```
java -jar target/*.jar
```

```
> git clone https://github.com/swayam614/ScientificCalculator-Devops.git
Cloning into 'ScientificCalculator-Devops'...
remote: Enumerating objects: 125, done.
remote: Counting objects: 100% (125/125), done.
remote: Compressing objects: 100% (81/81), done.
remote: Total 125 (delta 31), reused 114 (delta 20), pack-reused 0 (from 0)
Receiving objects: 100% (125/125), 23.54 KiB | 4.71 MiB/s, done.
Resolving deltas: 100% (31/31), done.
> cd ScientificCalculator-Devops
```

```
> mvn clean package
```

```
[INFO] Scanning for projects...
```

```
[INFO]
```

```
[INFO] -----< com.devops:scientific-calculator >-----
```

```
[INFO] Building scientific-calculator 1.0-SNAPSHOT
```

```
[INFO] from pom.xml
```

```
[INFO] -----[ jar ]-----
```

```
[INFO] -----
```

```
[INFO] BUILD SUCCESS
```

```
[INFO] -----
```

```
[INFO] Total time: 0.877 s
```

```
[INFO] Finished at: 2026-03-07T13:30:47+05:30
```

```
[INFO] -----
```

```
> java -jar target/*.jar
```

```
Scientific Calculator
```

```
1. Square Root ( $\sqrt{x}$ )
```

```
2. Factorial ( $x!$ )
```

```
3. Natural Log ( $\ln x$ )
```

```
4. Power ( $x^b$ )
```

```
5. Exit
```

```
Choose an option: |
```

7. Scientific Calculator Application Screenshots

The following screenshots show the working of the Scientific Calculator application with different operations.

<pre>Scientific Calculator 1. Square Root ($\sqrt{x}$) 2. Factorial ($x!$) 3. Natural Log ($\ln x$) 4. Power (x^b) 5. Exit Choose an option: 1 Enter number: 5 Result: 2.23606797749979</pre>	<pre>Scientific Calculator 1. Square Root ($\sqrt{x}$) 2. Factorial ($x!$) 3. Natural Log ($\ln x$) 4. Power (x^b) 5. Exit Choose an option: 2 Enter integer: 4 Result: 24</pre>
<pre>Scientific Calculator 1. Square Root ($\sqrt{x}$) 2. Factorial ($x!$) 3. Natural Log ($\ln x$) 4. Power (x^b) 5. Exit Choose an option: 3 Enter number: 2.76 Result: 1.0152306797290587</pre>	<pre>Scientific Calculator 1. Square Root ($\sqrt{x}$) 2. Factorial ($x!$) 3. Natural Log ($\ln x$) 4. Power (x^b) 5. Exit Choose an option: 4 Enter base: 2 Enter exponent: 4 Result: 16.0</pre>

8. Project Links

<https://github.com/swayam614/ScientificCalculator-Devops.git>

<https://hub.docker.com/repository/docker/swayam614/scientific-calculator/>

9. Summary for First-Time Users

To replicate this project:

- 1) Install Git, Maven, Jenkins, Docker, and Ansible on your system.
- 2) Clone the project repository from GitHub and configure Jenkins with repository access and Docker Hub credentials.
- 3) Push code changes to the GitHub repository.
- 4) Jenkins automatically triggers the pipeline to build, test, containerize, and deploy the application.
- 5) Run the Docker container in the terminal to start the CLI Scientific Calculator and perform operations interactively.